Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 4

на тему «Разработка бизнес-операций и обработки данных в GraphQL API»

Выполнил: Летко А.С.
студент группы 377901

Проверила: Сильванович Ю.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы – расширить разработанную ранее платёжную систему бизнес-операциями GraphQL API, реализовать обработку пользовательских действий и изменение состояния платежей, а также добавить фильтрацию, сортировку и агрегацию данных при сохранении изолированного взаимодействия компонентов через заглушки.

2 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

1.1 Исходная система и постановка задачи

В качестве основы использована система из предыдущей лабораторной работы. Её предметная область – обработка платежей: в системе представлены пользователи, продавцы, способы оплаты, платежи, счета, возвраты, транзакции, уведомления и статистика. Исходный API позволял получать связанные данные через GraphQL, а сведения хранились в PostgreSQL.

На данном этапе необходимо было перейти от преимущественно читающих запросов к бизнес-действиям. Для этого добавлены создание, завершение и отмена платежа; предусмотрены проверки входных данных и допустимости переходов между состояниями. Дополнительно расширены возможности выборки платежей и получения сводных показателей.

1.2 Изменения по сравнению с лабораторной работой 3

В предыдущей лабораторной работе были подготовлены структура проекта, база данных, GraphQL-запросы на чтение и локальный клиент-заглушка. В частности, список платежей уже поддерживал базовые фильтрацию и сортировку, но обработка этих параметров находилась в QueryController. Поэтому в настоящей работе они рассматриваются как развитие ранее созданного механизма, а не как полностью новая возможность.

В четвёртой работе в GraphQL-схему добавлены бизнес-операции createPayment, completePayment и cancelPayment, а также запрос динамической сводки paymentSummary. Обработка поиска платежей перенесена в PaymentService; фильтр дополнен идентификаторами клиента и продавца и валютой, добавлены проверки границ сумм и дат. Сервис выполняет проверки участников, суммы и состояния, а репозиторий сохраняет платёж и условно обновляет его статус.

Граница между компонентами сохранена: gateway передаёт вызовы через расширенный интерфейс ServiceClient и его локальную реализацию MockServiceClient. Для проверки сведений других областей PaymentService использует интерфейс PaymentParticipants и заглушку MockPaymentParticipants. Заглушки вызывают сервисные классы в том же

процессе, сетевой транспорт и gRPC отсутствуют. Общая конфигурация PostgreSQL и Docker Compose унаследована из предыдущей работы.

1.3Расширение GraphQL API

Схема расширена типом Mutation, входным типом CreatePaymentInput и результатом PaymentSummary. Mutation описывает действия с платежом, а не прямое изменение произвольных полей записи. Запрос payments принимает условия отбора и порядок выдачи; paymentSummary рассчитывает показатели по выбранным платежам. Контроллер принимает аргументы и передаёт выполнение через клиентскую границу, а правила операций сосредоточены в сервисе платежей.

1.4 Создание платежа и проверка бизнес-условий

Операция createPayment принимает идентификаторы клиента, продавца и способа оплаты, сумму, валюту и необязательное описание. До сохранения сервис проверяет, что клиент существует и имеет роль CLIENT, продавец активен, а способ оплаты активен и принадлежит указанному клиенту. Также проверяются обязательные поля, положительная сумма с точностью до двух знаков после запятой и трёхбуквенный код валюты.

После успешных проверок создаются уникальный идентификатор и время создания, а платежу присваивается начальный статус CREATED. Результат выполнения операции показан на рисунке 1.1: создан платёж на 30,5 BYN, и сервер вернул его идентификатор и начальное состояние.

Запрос, использованный для проверки, приведён в листинге 1.1.

Листинг 1.1 - Создание платежа

```
mutation {
  createPayment(input: {clientId: "u1",
    merchantId: "m1", paymentMethodId: "pm1",
    amount: 30.50, currency: "BYN",
    description: "Report demo"}) {
    id amount currency status
  }
}
```

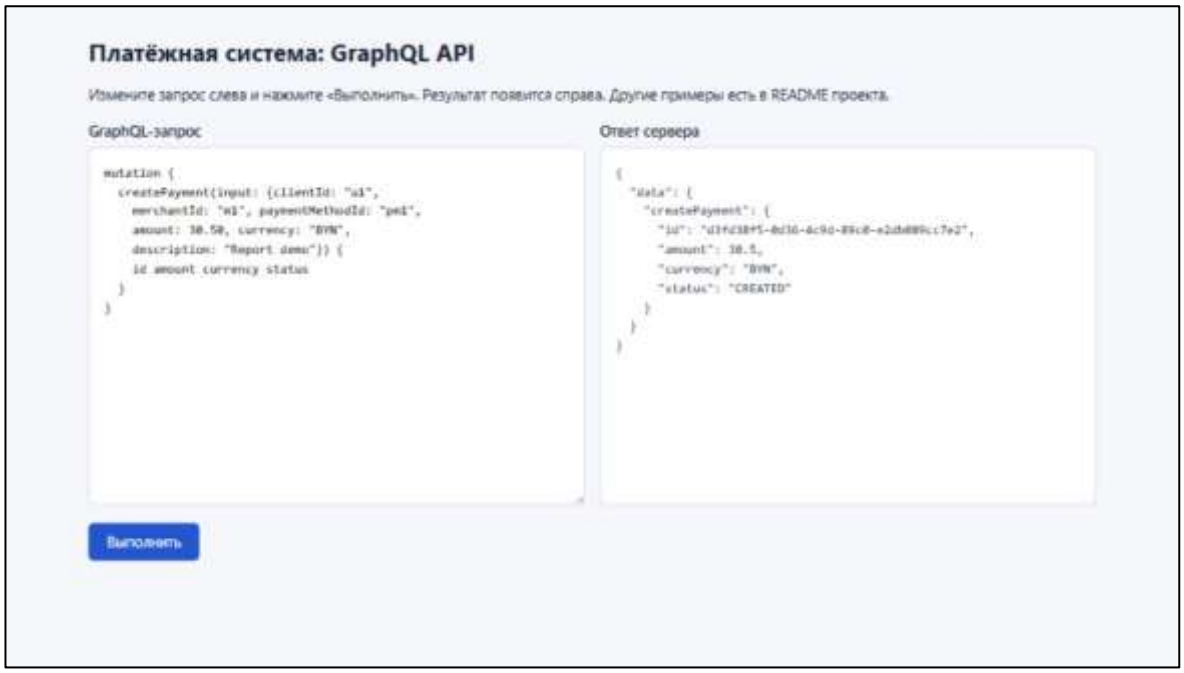


Рисунок 1.1 - Создание платежа

1.5 Завершение и отмена платежа

Операция completePayment переводит созданный платёж в состояние SUCCESS. На рисунке 1.2 показано завершение ранее созданного платежа: в ответе возвращается тот же идентификатор и новый статус. В учебной модели операция фиксирует изменение состояния; фактическое проведение денежных средств и изменение балансов счетов не реализованы.

Запрос, использованный для проверки, приведён в листинге 1.2. Идентификаторы созданных платежей в данном примере относятся к демонстрационной базе и при повторном создании будут другими.

Листинг 1.2 - Завершение платежа

```
mutation {
  completePayment(
    id: "d3fd38f5-0d36-4c9d-89c0-e2db089cc7e2"
  ) { id status amount currency }
}
```

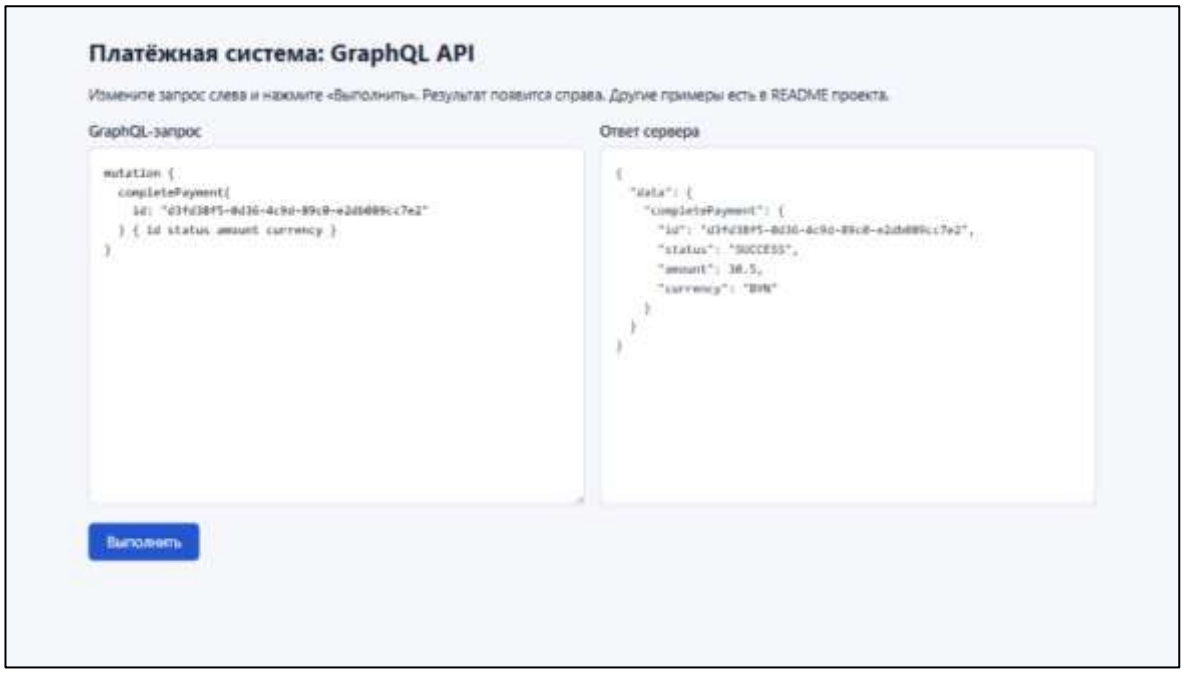


Рисунок 1.2 - Завершение созданного платежа

Отмена выполняется операцией cancelPayment и переводит платёж из CREATED в CANCELLED. Для демонстрации был создан отдельный платёж, после чего выполнена его отмена. Полученный статус показан на рисунке 1.3.

Запрос, использованный для проверки, приведён в листинге 1.3. Идентификаторы созданных платежей в данном примере относятся к демонстрационной базе и при повторном создании будут другими.

Листинг 1.3 - Отмена платежа

```
mutation {
  cancelPayment(
    id: "58a5ecb9-591a-4961-a15a-91988159aa27"
  ) { id status }
}
```

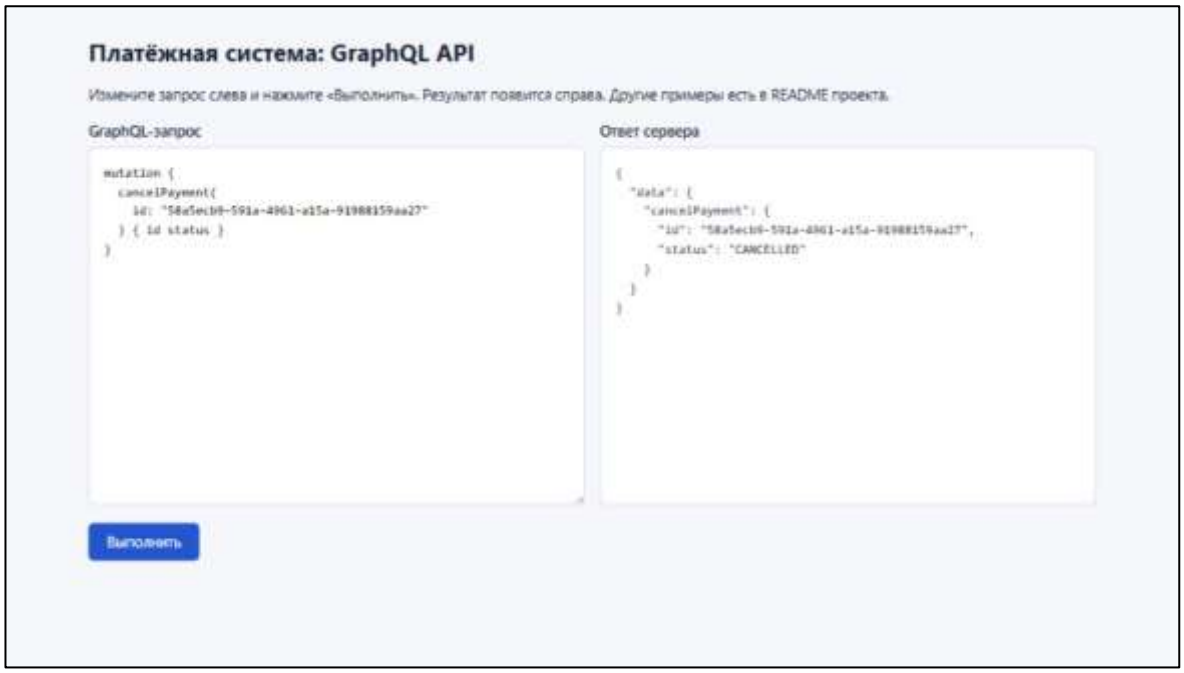


Рисунок 1.3 - Отмена созданного платежа

Повторное изменение уже завершённого или отменённого платежа недопустимо. На рисунке 1.4 представлена попытка отменить платёж после его завершения. Сервер возвращает сообщение об ошибке и классификацию BAD_REQUEST, а состояние платежа не меняется. Проверка перехода выполняется условным обновлением записи только при текущем статусе CREATED, что также предотвращает повторное успешное выполнение конкурирующих переходов.

Запрос, использованный для проверки, приведён в листинге 1.4. Идентификаторы созданных платежей в данном примере относятся к демонстрационной базе и при повторном создании будут другими.

Листинг 1.4 - Попытка отмены завершённого платежа

```
mutation {
  cancelPayment(
    id: "d3fd38f5-0d36-4c9d-89c0-e2db089cc7e2"
  ) { id status }
}
```

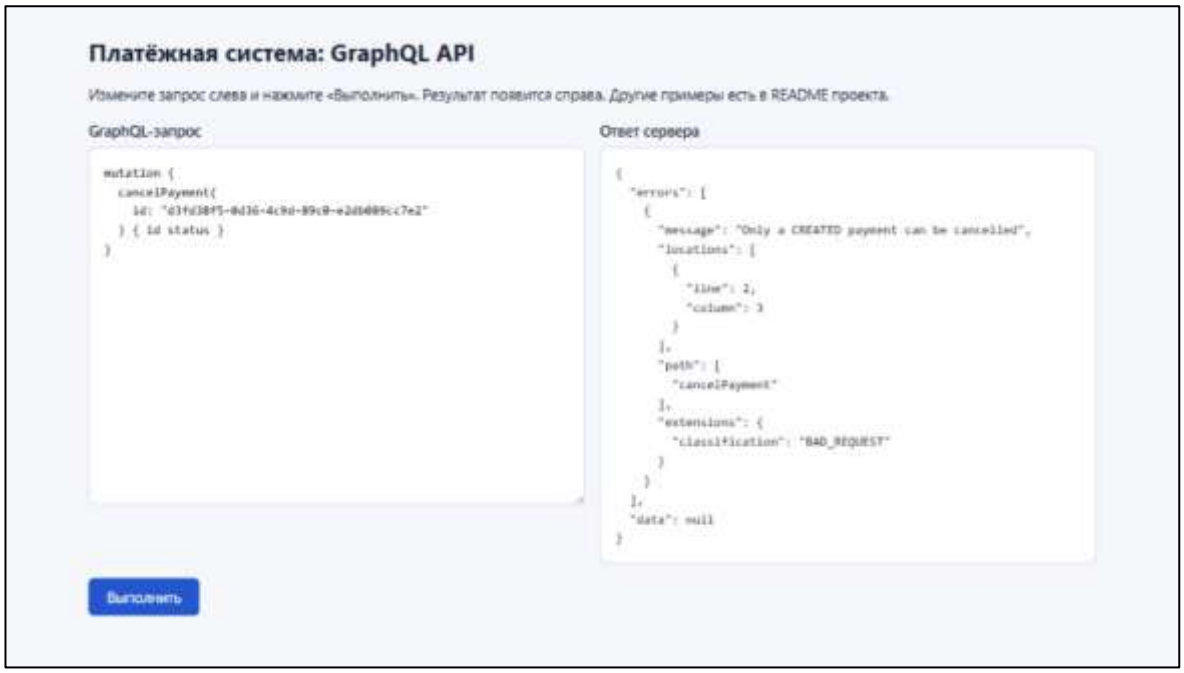


Рисунок 1.4 - Обработка недопустимого перехода состояния

1.6 Фильтрация и сортировка платежей

Запрос payments поддерживает фильтрацию по клиенту, продавцу, валюте, статусу, минимальной и максимальной сумме, а также по начальной и конечной дате. Границы диапазонов включаются. Дату можно задать календарным днём в UTC или моментом времени в формате ISO-8601. Сервис проверяет корректность дат и порядок границ диапазонов.

Для сортировки предусмотрены поля CREATED_AT и AMOUNT, а также направления ASC и DESC. На рисунке 1.5 выбраны успешные платежи продавца m1 в BYN и задана сортировка по убыванию суммы. В ответе сначала расположен платёж на 125,5 BYN, затем – на 30,5 BYN, что соответствует указанному порядку.

Запрос, использованный для проверки, приведён в листинге 1.5.

Листинг 1.5 - Фильтрация и сортировка платежей

```
query {
  payments(filter: {merchantId: "m1",
    status: SUCCESS, currency: "BYN"},
    sort: {field: AMOUNT, direction: DESC}) {
    amount currency status
  }
}
```

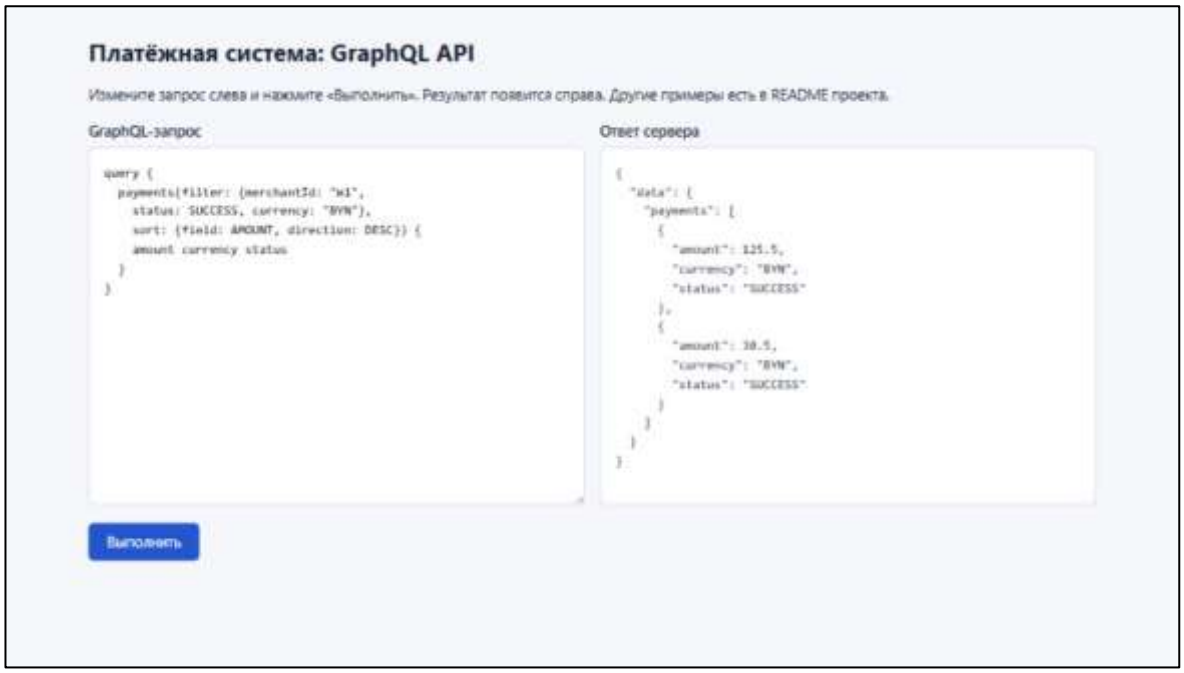


Рисунок 1.5 - Фильтрация и сортировка платежей

1.7 Агрегация данных

Запрос paymentSummary применяет те же условия отбора и возвращает количество платежей, общую сумму и валюту. Агрегация выполняется по фактическим данным платежей, а не по заранее заполненным демонстрационным записям статистики. Платежи разных валют не суммируются: если в выборке обнаружено несколько валют, необходимо уточнить фильтр.

Результат вычисления для успешных платежей продавца m1 в BYN приведён на рисунке 1.6. В выбранный момент времени найдены два платежа на общую сумму 156 BYN. Без фильтра статуса сводка включает платежи всех состояний, в том числе отменённые; при пустой выборке количество и сумма равны нулю.

Запрос, использованный для проверки, приведён в листинге 1.6.

Листинг 1.6 - Получение сводки по платежам

```
query {
  paymentSummary(filter: {merchantId: "m1",
    status: SUCCESS, currency: "BYN"}) {
    count totalAmount currency
  }
}
```

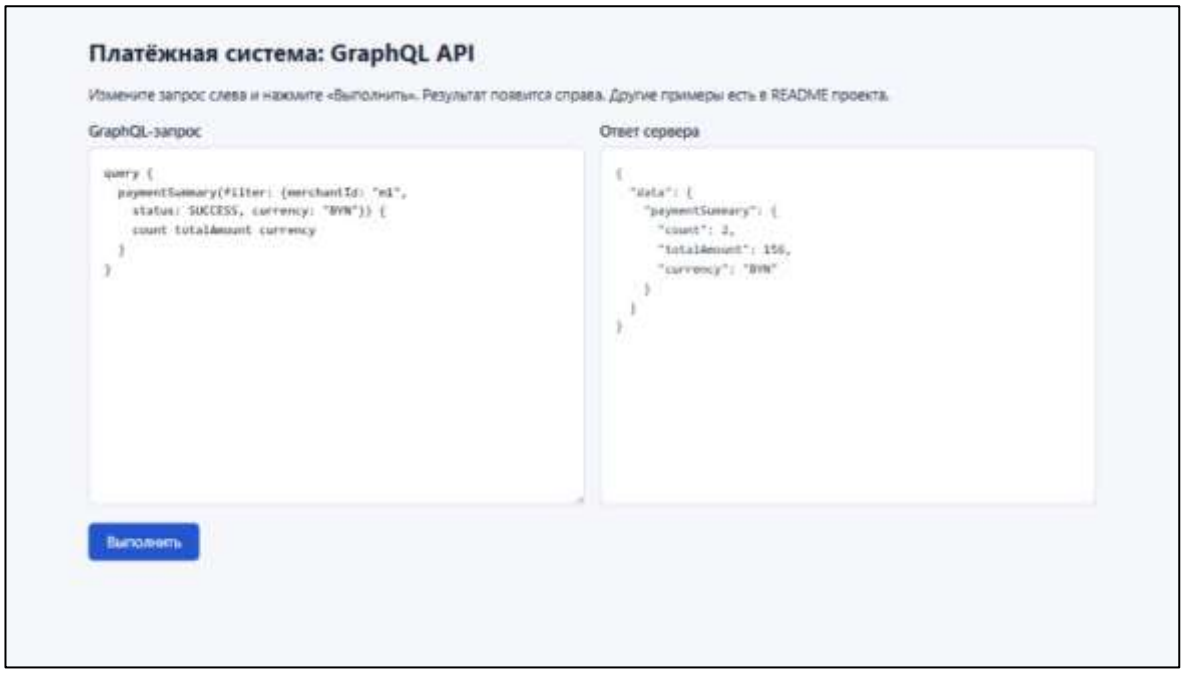


Рисунок 1.6 - Сводка по выбранным платежам

1.8 Обработка некорректных данных

При нарушении бизнес-условий сервис формирует исключение с пояснением, а обработчик GraphQL преобразует его в ответ с разделом errors и классификацией BAD_REQUEST. Это позволяет клиенту получить понятную причину отказа. На рисунке 1.7 показана проверка отрицательной суммы: создание платежа отклонено, так как сумма должна быть положительной и соответствовать допустимой точности.

Запрос, использованный для проверки, приведён в листинге 1.7.

Листинг 1.7 - Проверка отрицательной суммы

```
mutation {
  createPayment(input: {clientId: "u1",
    merchantId: "m1", paymentMethodId: "pm1",
    amount: -5, currency: "BYN"}) { id }
}
```

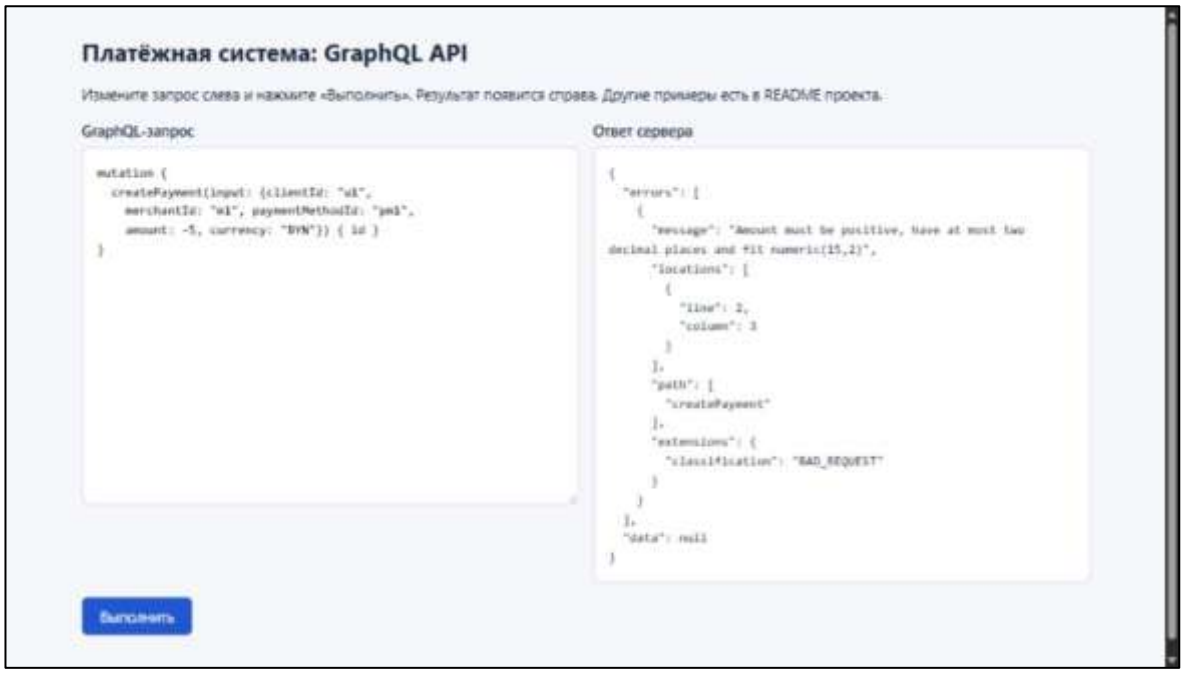


Рисунок 1.7 - Проверка некорректной суммы платежа

Также предусмотрены сообщения об отсутствующем клиенте, неактивном продавце, неподходящем способе оплаты, неверном коде валюты, отсутствии платежа и недопустимом переходе состояния. Синтаксис GraphQL и обязательность аргументов дополнительно проверяются на уровне схемы до выполнения бизнес-операции.

1.9 Итоги проверки

В работающем приложении последовательно проверены получение платежа со связанными данными, создание платежей, успешное завершение, отмена, отказ при повторном изменении состояния, фильтрация, сортировка, вычисление сводки и отказ при отрицательной сумме. Значения на рисунках отражают состояние демонстрационной базы в момент проверки и могут отличаться при последующих запусках с сохранённым томом.

1.10 Последовательность выполнения операции создания

При выполнении createPayment запрос сначала поступает в GraphQL-подсистему приложения. Схема определяет обязательные аргументы и тип результата: клиент передаёт объект CreatePaymentInput, а в ответе выбирает нужные поля Payment. Если обязательный аргумент отсутствует или тип значения не соответствует схеме, запрос отклоняется на этапе проверки GraphQL и бизнес-операция не выполняется.

После успешной проверки схемы QueryController передаёт входные данные методу createPayment интерфейса ServiceClient. MockServiceClient

делегирует вызов методу create сервиса PaymentService. Контроллер не сохраняет запись самостоятельно и не определяет правила допустимости платежа: эти действия выполняются во внутреннем сервисе и его репозитории.

PaymentService извлекает идентификаторы участников, сумму и валюту. Обязательные строковые значения проверяются на отсутствие и пустое содержимое, затем из них удаляются крайние пробелы. Валюта приводится к верхнему регистру. Проверка валюты ограничена формой записи: принимается последовательность из трёх латинских букв; сверка с официальным перечнем валют не выполняется.

Сумма преобразуется в BigDecimal, что позволяет выполнять проверки и вычисления с десятичным представлением на стороне сервиса. Она должна быть больше нуля, содержать не более двух знаков после запятой и помещаться в столбец numeric(15,2). При этом во внешней GraphQL-схеме используется Float, поэтому для промышленной платёжной системы целесообразно определить специальный точный десятичный тип и дополнительно рассмотреть порядок преобразования входных значений.

Далее через PaymentParticipants запрашиваются данные клиента, продавца и способа оплаты. Проверяется роль клиента, статус продавца и принадлежность активного способа оплаты клиенту. Если хотя бы одно условие нарушено, выбрасывается ошибка и вставка не выполняется. После успешной проверки формируются UUID и время создания в UTC, репозиторий добавляет запись со статусом CREATED, а сервис возвращает сохранённый платёж.

1.11 Реализация изменения состояния и доступа к данным

В модели предусмотрены три состояния: CREATED, SUCCESS и CANCELLED. Начальное состояние назначается при создании; из него доступны переходы к успешному завершению и отмене. Переходы между SUCCESS и CANCELLED, возврат в CREATED и повторное применение операции не предусмотрены. Таким образом, система не позволяет случайно изменить уже завершённый пользовательский сценарий.

Метод transition репозитория выполняет параметризованный SQL-запрос с двумя условиями: совпадением идентификатора и текущим статусом CREATED. Если обновлена одна запись, сервис повторно считывает платёж и возвращает результат. Если число обновлённых записей равно нулю, сервис отдельно проверяет наличие идентификатора и сообщает либо об отсутствующем платеже, либо о недопустимом текущем состоянии.

Проверка состояния включена непосредственно в UPDATE, а не выполняется только предварительным чтением. При одновременных попытках изменить один созданный платёж условие позволит успешно обновить его только до тех пор, пока он находится в CREATED. Следующая попытка не сможет применить тот же переход к уже изменённой записи. Такой

подход решает задачу атомарности отдельного изменения статуса в текущей модели.

Репозиторий использует JdbcTemplate и параметры запроса для передачи значений отдельно от текста SQL. Он обращается к таблице payment.payments и не принимает решения о том, является ли клиент допустимым или может ли платёж быть завершён: эти правила задаёт PaymentService. Изменение статуса не создаёт новые записи транзакций, уведомлений или статистики и не изменяет счета.

1.12 Обработка параметров поиска

Для каждого необязательного параметра фильтра сервис проверяет соответствующее условие только тогда, когда параметр передан. Если одновременно указаны несколько условий, платёж должен удовлетворять каждому из них. Например, при передаче продавца, статуса и валюты результат содержит записи этого продавца с заданными статусом и валютой.

Минимальная и максимальная суммы включаются в диапазон: сумма записи должна быть не меньше нижней и не больше верхней границы. Оба значения проходят ту же проверку положительности и точности, что и сумма платежа. Если минимальное значение больше максимального, сервис возвращает ошибку вместо выдачи некорректного результата.

Дата в формате YYYY-MM-DD трактуется как календарный день UTC. Для начальной границы используется начало дня, для конечной – его последний момент. Если передан полный момент времени, он разбирается как Instant. Неверный формат или начальная дата позже конечной приводят к ошибке. При сравнении значения непосредственно на допустимых границах включаются в результат.

Сортировка выполняется по сумме либо времени создания. Направление DESC меняет порядок на обратный; по умолчанию используется дата создания по возрастанию. Если значения основного поля одинаковы, дополнительное сравнение идентификаторов делает порядок стабильным. В текущем учебном варианте репозиторий считывает список платежей целиком, после чего сервис фильтрует и сортирует его в памяти. Для больших объёмов данных потребуется перенести отбор и сортировку в SQL и добавить постраничную выдачу.

1.13 Особенности вычисления сводных показателей

Метод summary использует ту же операцию поиска, что и список платежей. Это означает, что условия отбора для списка и сводки интерпретируются одинаково: например, при фильтре status SUCCESS учитываются только успешно завершённые платежи. Далее сервис определяет

набор валют полученных записей, считает их количество и складывает суммы с помощью BigDecimal.

Если в выборке присутствует более одной валюты, сумма не возвращается и формируется ошибка с предложением указать валюту. Если записей нет, количество и сумма равны нулю; поле валюты содержит значение фильтра, приведённое к верхнему регистру, либо пустую строку при отсутствии фильтра. Такой результат позволяет отличить пустую выборку от ошибки вычисления.

Сводка paymentSummary и ранее существовавшие запросы analytics и reports имеют разное назначение. Первая вычисляется по текущим платежам в момент обращения. Последние читают заранее подготовленные записи статистики и не пересчитываются автоматически после создания, отмены или завершения платежа. В частности, paymentSummary не вычитает возвраты и не является расчётом чистой выручки.

1.14 Сопоставление результата с заданием

Требование расширить GraphQL API выполнено добавлением трёх Mutation и сводного Query, входных структур и дополнительных параметров фильтрации. Операции отражают пользовательские действия: оформление платежа, его завершение и отмену. Список платежей поддерживает несколько условий отбора и два поля сортировки, а paymentSummary предоставляет количество и сумму отобранных записей.

Требование реализовать бизнес-операции выполнено за счёт проверки участников, принадлежности способа оплаты, суммы, валюты и допустимых переходов состояния. Некорректные действия возвращают ошибки GraphQL. Сервисный слой содержит эти правила, обращается к репозиторию и подготавливает данные для внешнего слоя.

Требование сохранить изолированное взаимодействие выполнено в границах учебной архитектуры через интерфейсы и локальные заглушки. При этом изоляция является логической: модули не запускаются отдельно и используют общую базу. Реальные gRPC-вызовы не выполняются; перед внедрением сетевого взаимодействия потребуется отдельно определить контракты, развёртывание и обработку сетевых отказов.

Проверки на рисунках 1.1–1.7 подтверждают показанные сценарии на конкретном наборе данных. Проверки не являются исчерпывающим автоматизированным тестированием всех сочетаний параметров и конкурентных ситуаций. Например, описания обработки неверного диапазона дат и разных валют основаны на анализе реализации; отдельные результаты их выполнения на рисунках не представлены.

3 ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы платёжная система расширена бизнес-операциями GraphQL API. Реализованы создание, завершение и отмена платежей с проверкой входных данных и допустимости изменения состояния; добавлены фильтрация, сортировка и вычисление сводных показателей. Основная логика сосредоточена во внутреннем сервисе платежей, а взаимодействие между компонентами организовано через локальные заглушки без gRPC. Работоспособность основных сценариев продемонстрирована запросами и ответами приложения.